UNIVERSIDADE FEDERAL DE ALAGOAS
CAMPUS PRAÇA OSÓRIO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: RELATÓRIO TÉCNICO, PERSISTÊNCIA DE DADOS E HISTÓRICO DE DECISÕES

Curitiba
2026

ANA JÚLIA NICOLACK
YASMIN DA ROCHA LOPES

PLAYDEX: RELATÓRIO TÉCNICO, PERSISTÊNCIA DE DADOS E HISTÓRICO DE DECISÕES

Relatório técnico apresentado à disciplina de Desenvolvimento de Aplicativos Móveis do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Positivo, como requisito parcial à avaliação da atividade prática.

Orientador: Prof. Dr. André Jeller

Curitiba
2026

RESUMO

O presente relatório técnico documenta a evolução arquitetural e a implementação da persistência de dados no aplicativo móvel PlayDex, desenvolvido no âmbito da disciplina de Desenvolvimento de Aplicativos Móveis do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas. O objetivo principal do estudo consistiu na transição de um protótipo com armazenamento temporário em memória RAM (`mutableStateListOf`) para uma estrutura persistente e reativa embasada no banco de dados relacional SQLite, utilizando a biblioteca Room. O percurso metodológico contemplou a reestruturação da camada de apresentação mediante a adoção do padrão MVVM com `AndroidViewModel`, o mapeamento de entidades relacionais (`ItemPesquisa` e `ItemFavorito`), a implementação de interfaces de acesso a dados (DAOs) com suporte a operações assíncronas (*Coroutines*), e o gerenciamento explícito de *threads* de execução (`Dispatchers.IO` e `Dispatchers.Main`). Como resultados, alcançou-se a retenção definitiva de dados de histórico de pesquisas e avaliações de jogos, eliminando a volatilidade do estado perante o encerramento do aplicativo ou alterações na orientação da tela (*Activity lifecycle*). Conclui-se que a combinação do Room com o desacoplamento promovido pelas *ViewModels* garante a estabilidade, a escalabilidade e a reatividade necessárias para aplicações Android modernas construídas com Jetpack Compose.

Palavras-chave: persistência de dados; Room database; AndroidViewModel; Jetpack Compose; desenvolvimento móvel.

SUMÁRIO

1 INTRODUÇÃO.....14
2 EVOLUÇÃO DO PROJETO: DO TRABALHO 2 AO TRABALHO 3.....15
3 ESTRUTURA DO BANCO DE DADOS LOCAL: DATABASE, ENTITIES E DAOS16
3.1 Database.....16
3.2 Entidades.....16
3.3 Objeto de Acesso a Dados (DAOS).....17
5 DA LISTA EM MEMÓRIA AO BANCO DE DADOS PERSISTENTE.....22
5.1 Mudança Conceitual: mutableStateListOf versus Room.....22
5.2 Desafios Técnicos e Soluções Adotadas.....22
6 DEMONSTRAÇÃO DO APLICATIVO EM FUNCIONAMENTO.....25
6.1 Tela de Pesquisa e Histórico de Buscas.....25
6.2 Tela de Favoritos e Gerenciamento de Análises.....25
6.3 Validação da Persistência de Dados e do Ciclo de Vida.....25
Vídeo geral do Projeto.mp4.....26
7 DIFICULDADES ENCONTRADAS E RESOLUÇÃO DE PROBLEMAS.....27
7.1 Gerenciamento de Estado e Ciclo de Vida: Transição para AndroidViewModel.....27
7.2 Adequação ao Fluxo Assíncrono (Dispatchers.IO e Dispatchers.Main).....27
8 CONSIDERAÇÕES FINAIS.....29

1 INTRODUÇÃO

O avanço no desenvolvimento de aplicações para dispositivos móveis exige a adoção de arquiteturas robustas capazes de conciliar interfaces reativas com mecanismos eficientes de persistência de dados local. No ecossistema Android, a preservação do estado da aplicação perante o ciclo de vida das *Activities* e a retenção permanente de informações inseridas pelo usuário representam requisitos fundamentais para assegurar a usabilidade e a estabilidade dos sistemas.

Este relatório técnico aborda o processo de evolução do aplicativo PlayDex, uma plataforma desenvolvida em Kotlin voltada para o gerenciamento de pesquisas e avaliações de jogos eletrônicos (*reviews*). O trabalho situa-se na transição entre o estágio inicial do projeto (Trabalho 2), caracterizado pela dependência de estruturas voláteis em memória RAM (`mutableStateListOf`), e o estado atual (Trabalho 3), estruturado sobre o banco de dados relacional SQLite integrado via biblioteca Room e com suporte a *AndroidViewModel*.

O objetivo geral deste documento é detalhar o histórico de decisões arquiteturais, o mapeamento das entidades e interfaces DAO, bem como as estratégias adotadas para contornar os desafios de assincronismo e gerenciamento de *threads* impostos pela manipulação de banco de dados no Jetpack Compose.

2 EVOLUÇÃO DO PROJETO: DO TRABALHO 2 AO TRABALHO 3

Para a consolidação da versão atual do aplicativo, foram realizadas reformulações lógicas e estruturais significativas na aplicação em relação ao estágio apresentado no Trabalho 2.

No estado anterior (Trabalho 2), o código-fonte encontrava-se concentrado em um único diretório, no pacote raiz (**`com.example.my application`**), compartilhando no mesmo nível a navegação (**`Rotas.kt`**), a classe principal (**`MainActivity.kt`**) e as telas da interface do usuário (**`TelaPrincipal.kt`**, **`TelaPesquisa.kt`**, **`TelaReview.kt`**, entre outras). Adicionalmente, a persistência de dados era volátil, baseada no armazenamento em memória RAM por meio de listas temporárias (**`mutableStateListOf`**), o que resultava na perda integral das informações ao encerrar o aplicativo. Essa abordagem acarretava instabilidade no gerenciamento de estado, causando reinicializações da interface e perda de alterações não salvas mediante eventos de rotação de tela ou navegação entre rotas.

No estado atual (Trabalho 3), embora os arquivos de código tenham sido mantidos no pacote raiz (**`com.example.myapplication`**), a lógica interna e a arquitetura dos componentes foram completamente reformuladas. A persistência de dados foi implementada de forma definitiva via banco de dados local SQLite por meio da biblioteca Room, mediante a criação de entidades, objetos de acesso a dados (**`FavoritoDao.kt`**, **`PesquisaDAO.kt`**), a classe de banco de dados (**`AppDatabase.kt`**) e utilitários de manipulação (**`OperacoesBanco.kt`**), garantindo o salvamento permanente de avaliações, pesquisas e favoritos no armazenamento interno do dispositivo. Por fim, introduziu-se o gerenciamento de estado via *ViewModels* (**`FavoritosViewModel.kt`**, **`PesquisaViewModel.kt`**), responsável pela lógica de negócios e pelo acompanhamento do ciclo de vida da interface, assegurando a retenção das informações durante a navegação ou a rotação do dispositivo.

3 ESTRUTURA DO BANCO DE DADOS LOCAL: *DATABASE*, *ENTITIES* E *DAOS*

Para fundamentar a persistência de dados no aplicativo PlayDex, implementou-se um banco de dados local SQLite baseado na biblioteca Room, estruturado a partir dos componentes *Database*, *Entities* e *Data Access Objects* (DAOs).

3.1 Database

A classe **`AppDatabase`** consiste em uma classe abstrata anotada com **`@Database`** que estende **`RoomDatabase`**, atuando como o ponto central de acesso ao banco SQLite local (**`app_database`**). Essa estrutura é responsável por registrar as entidades **`ItemPesquisa`** e **`ItemFavorito`**, definir a versão da base de dados (**`version = 2`**) e prover os métodos abstratos para obter as instâncias dos DAOs (**`pesquisaDao()`** e **`favoritoDao()`**). Ademais, emprega-se o padrão *Singleton* (**`INSTANCE`**), assegurando a existência de uma única instância do banco de dados em toda a aplicação.

3.2 Entidades

As entidades da base de dados foram mapeadas conforme as regras de negócio do sistema:

- **`ItemFavorito`** (Tabela: **`tabela_favoritos`**): Mapeia os jogos armazenados na biblioteca do usuário. É composta pelos campos **`id`** (*Int*, com **`@PrimaryKey(autoGenerate = true)`** para identificação numérica única), **`title`** (*String*, contendo o nome do jogo), **`status`** (*String*, indicando o estado do jogo, como "Jogando" ou "Zerado"), **`review`** (*String*, armazenando a resenha textual), **`nota`** (*Int*, com a avaliação numérica) e **`data`** (*String*, registrando o momento da avaliação). A escolha desses campos fundamenta-se nas necessidades de um diário de jogos: o **`id`** auto incrementado garante a unicidade do registro e permite avaliações distintas de um mesmo título em momentos variados, enquanto os demais atributos armazenam o progresso e a análise do usuário para exibição nas telas do sistema.
- **`ItemPesquisa`** (Tabela: **`historico_pesquisa`**): Representa os registros do histórico de buscas. Estruturada pelos campos **`id`** (*Int*, com **`@PrimaryKey(autoGenerate = true)`**) e **`nome`** (*String*, contendo o termo digitado na pesquisa). A definição dessa estrutura busca criar um modelo leve para o

histórico, onde a chave primária auto incrementada impede conflitos ao salvar termos repetidos e o campo **`nome`** possibilita a rápida recuperação dos itens consultados.

3.3 Objeto de Acesso a Dados (DAOS)

Por fim, as operações de manipulação do banco são intermediadas pelas interfaces DAO. A **`FavoritoDao`** gerencia a tabela **`tabela_favoritos`**, contendo as operações completas de CRUD: inserção (**`@Insert suspend fun inserir(...)`**), consulta ordenada do registro mais recente ao mais antigo (**`@Query("SELECT * FROM tabela_favoritos ORDER BY id DESC") suspend fun buscarFavoritos()`**), busca por identificador (**`@Query("SELECT * FROM tabela_favoritos WHERE id = :id") suspend fun buscarPorId(...)`**), atualização (**`@Update suspend fun atualizarFavorito(...)`**) e remoção (**`@Delete suspend fun removerFavorito(...)`**). Paralelamente, a **`PesquisaDAO`** administra a tabela **`historico_pesquisa`**, fornecendo métodos para inserção (**`@Insert fun inserir(...)`**), consulta em ordem decrescente (**`@Query("SELECT * FROM historico_pesquisa ORDER BY id DESC") fun buscarItens()`**), alteração (**`@Update fun atualizarItem(...)`**) e deleção de termos do histórico (**`@Delete fun removerItem(...)`**).

4 ESTRUTURA DAS *QUERIES* E OPERAÇÕES DOS DAOS

Para o desenvolvimento das interfaces **`FavoritoDao.kt`** e **`PesquisaDAO.kt`**, as consultas (*queries*) e operações do banco de dados foram projetadas priorizando o assincronismo, a integridade dos dados e o alinhamento com as necessidades da interface do usuário do PlayDex.

4.1 Estrutura Operacional e Assincronismo (*Coroutines*)

A execução das operações de banco de dados foi estruturada de forma a evitar o bloqueio da *thread* principal (*UI thread*), garantindo a fluidez da interface:

- **`FavoritoDao:`** Todos os métodos declarados utilizam a palavra-chave **`suspend`** (como **`suspend fun inserir`** e **`suspend fun buscarFavoritos`**). Essa abordagem indica o processamento assíncrono por meio de *Coroutines* do Kotlin, executando as instruções de leitura e escrita em *threads* secundárias.

```kotlin
package com.example.myapplication

import androidx.room3.Dao
import androidx.room3.Delete
import androidx.room3.Insert
import androidx.room3.Query
import androidx.room3.Update

@Dao
interface FavoritoDao {
    @Insert
    suspend fun inserir(favorito: ItemFavorito)

    @Query("SELECT * FROM tabela_favoritos ORDER BY id DESC")
    suspend fun buscarFavoritos(): List<ItemFavorito>

    @Query("SELECT * FROM tabela_favoritos WHERE id = :id")
    suspend fun buscarPorId(id: Int): ItemFavorito?
    @Update
    suspend fun atualizarFavorito(favorito: ItemFavorito)

    @Delete
    suspend fun removerFavorito(favorito: ItemFavorito)
}
```

- **`PesquisaDAO`:** Os métodos declarados na interface apresentam assinatura síncrona; contudo, o gerenciamento do assincronismo é delegado à camada intermediária de operações (**`OperacoesBanco.kt`**). Nessa camada, as chamadas utilizam funções **`suspend`** encapsuladas em blocos de tratamento de exceções (**`try-catch`**), assegurando o controle de erros durante o acesso aos dados.

```kotlin
package com.example.myapplication

import androidx.room3.Dao
import androidx.room3.Delete
import androidx.room3.Insert
import androidx.room3.Query
import androidx.room3.Update

@Dao
interface PesquisaDAO {
    @Insert
    fun inserir(item: ItemPesquisa)

    @Query("SELECT * FROM historico_pesquisa ORDER BY id DESC")
    fun buscarItens(): List<ItemPesquisa>

    @Update
    fun atualizarItem(item: ItemPesquisa)

    @Delete
    fun removerItem(item: ItemPesquisa)
}
```

4.2 Operações Fundamentais (CRUD)

Ambas as interfaces de acesso a dados contemplam a estrutura completa do ciclo CRUD (*Create, Read, Update, Delete*):

- **Inclusão (*Create*):** Mapeada pela anotação **`@Insert`** em ambas as interfaces (**`inserir`**).
- **Atualização (*Update*):** Mapeada pela anotação **`@Update`** (**`atualizarFavorito`** em **`FavoritoDao`** e **`atualizarItem`** em **`PesquisaDAO`**).
- **Exclusão (*Delete*):** Mapeada pela anotação **`@Delete`** (**`removerFavorito`** em **`FavoritoDao`** e **`removerItem`** em **`PesquisaDAO`**).

- **Consulta Geral (*Read*):** Mapeada pela anotação **`@Query`**, retornando todos os registros ordenados de forma decrescente pelo identificador (**`SELECT * FROM ... ORDER BY id DESC`**).

4.3 Consultas Específicas e Justificativas Arquiteturais

Além das operações padrão de manipulação global, foram implementadas *queries* direcionadas a casos de uso específicos da aplicação:

- **Consulta Específica por Identificador (`buscarPorId`):**
  - *Query:*

```
@Query("SELECT * FROM tabela_favoritos WHERE id = :id")
```

  - *Assinatura:*

```
suspend fun buscarPorId(id: Int): ItemFavorito?
```

  - *Justificativa:* Este método viabiliza o resgate direto de um registro individual a partir de sua chave primária (**`id`**). Sua implementação evita o tráfego desnecessário de toda a lista de favoritos na memória, sendo utilizado para carregar a tela de detalhes ou de edição de uma análise de jogo específica já cadastrada.
- **Ordenação Cronológica Inversa (`ORDER BY id DESC`):**
  - *Query em **`FavoritoDao`**:*

```
@Query("SELECT * FROM tabela_favoritos ORDER BY id DESC")
```

  - *Query em **`PesquisaDAO`**:*

```
@Query("SELECT * FROM historico_pesquisa ORDER BY id DESC")
```

  - *Justificativa:* Como os identificadores (**`id`**) são gerados de forma auto incrementada pelo Room, a ordenação decrescente garante que os registros

mais recentes sejam posicionados no topo das listagens, atendendo ao comportamento esperado na exibição do histórico de pesquisas e da biblioteca de favoritos.

5 DA LISTA EM MEMÓRIA AO BANCO DE DADOS PERSISTENTE

A migração de uma arquitetura baseada em estado temporário para uma solução de persistência relacional exigiu adequações conceituais e estruturais no aplicativo PlayDex, conforme abordado a seguir.

5.1 Mudança Conceitual: *mutableStateListOf* versus Room

A estrutura **`mutableStateListOf`** consiste em uma lista reativa em memória RAM, nativa do Jetpack Compose. Qualquer alteração em seu conteúdo dispara atualizações imediatas na interface do usuário (*UI thread*) sem a necessidade de conversão de tipos ou gerenciamento de *threads*. Contudo, sua limitação fundamenta-se na volatilidade dos dados, que são completamente perdidos ao encerrar a aplicação.

Em contrapartida, a biblioteca Room provê uma camada de abstração sobre o banco de dados relacional SQLite, garantindo a persistência definitiva no armazenamento do dispositivo. Essa mudança exige um mapeamento estruturado em entidades (**`@Entity`**), definição explícita de chaves primárias (**`@PrimaryKey`**), execução assíncrona por meio de *Coroutines* (**`suspend`**) e abstração das operações através de interfaces DAO.

5.2 Desafios Técnicos e Soluções Adotadas

Durante o processo de transição entre o modelo em memória e o banco de dados local, foram enfrentados e superados quatro desafios estruturais principais:

- **Perda da Reatividade Automática Direta (Listas Estáticas versus Fluxo de Dados):**
    - *Desafio:* A substituição do **`mutableStateListOf`** por chamadas diretas ao DAO que retornam listas estáticas (como **`fun buscarItens(): List<ItemPesquisa>`** ou **`suspend fun buscarFavoritos(): List<ItemFavorito>`**) eliminou a notificação automática de mudanças para a interface. O retorno de uma lista comum do banco não notifica o Compose sobre novas inserções ou deleções.
    - *Solução:* Desenvolveu-se uma camada intermediária de funções auxiliares no arquivo **`OperacoesBanco.kt`.** Essa estrutura é responsável por disparar o

recarregamento explícito dos dados e atualizar o estado da tela sempre que uma operação de gravação ou exclusão é executada.

- **Bloqueio da *Thread* Principal (Operações de I/O na *UI Thread*):**
  - *Desafio:* Por padrão, o Room impede que operações de leitura e escrita sejam executadas na *thread* principal para evitar travamentos (*lag/jank*) na interface do usuário.
  - *Solução:* No **`FavoritoDao`**, os métodos foram declarados com a palavra-chave **`suspend`**, permitindo a chamada assíncrona dentro de escopos de corrotinas (**`viewModelScope`** ou **`rememberCoroutineScope`**). Ademais, no arquivo **`OperacoesBanco.kt`**, as chamadas ao DAO foram encapsuladas em blocos **`suspend`** com tratamento de exceções (**`try-catch`**), assegurando a captura de falhas em logs sem a interrupção abrupta da aplicação.

- **Identificação e Manipulação dos Registros (Gerenciamento de Chaves Primárias):**
  - *Desafio:* No modelo anterior com **`mutableStateListOf`**, os objetos eram manipulados e removidos diretamente pela referência da instância na memória. A migração para o banco relacional passou a exigir um identificador exclusivo para cada registro na tabela SQLite.
  - *Solução:* Configurou-se o campo **`id`** com a anotação **`@PrimaryKey(autoGenerate = true)`** nas entidades **`ItemFavorito`** e **`ItemPesquisa`**. Ao instanciar novos objetos, atribui-se o valor inicial **`id = 0`**, delegando ao Room a geração automática de chaves primárias únicas e incrementais, o que viabiliza consultas direcionadas (**`WHERE id = :id`**) e remoções precisas via **`@Delete`**.

- **Versionamento da Base de Dados (*Database Migration*):**
  - *Desafio:* A inclusão de novas entidades e tabelas na estrutura (como a incorporação da **`tabela_favoritos`** ao lado de **`historico_pesquisa`**) exige a atualização do esquema do banco. A inexistência de um controle de versão adequado causa falhas críticas (*crashes*)

por desconformidade de esquema em dispositivos com instâncias prévias do banco.

- *Solução:* A anotação da classe principal do banco de dados foi atualizada para **`@Database(entities = [ItemPesquisa::class, ItemFavorito::class], version = 2)`**, registrando-se também o novo método abstrato **`abstract fun favoritoDao(): FavoritoDao`** na classe **`AppDatabase`**.

6 DEMONSTRAÇÃO DO APLICATIVO EM FUNCIONAMENTO

Nesta seção, apresenta-se a demonstração prática do aplicativo PlayDex em ambiente de execução. O registro audiovisual ilustra a navegação entre as telas, a reatividade da interface desenvolvida em Jetpack Compose e a integração com o banco de dados local Room.

Durante a execução da aplicação, observam-se em tempo real as seguintes operações e fluxos de dados:

6.1 Tela de Pesquisa e Histórico de Buscas

- **Inserção e Persistência:** Entrada e inclusão de novos termos na barra de busca, resultando na gravação imediata do registro no banco de dados SQLite por meio da tabela **`historico_pesquisa`**.
- **Atualização Reativa:** Atualização dinâmica da listagem de histórico, apresentada em ordem cronológica decrescente (do termo mais recente ao mais antigo).
- **Remoção de Registros:** Exclusão individual de itens do histórico de pesquisas com sincronização instantânea da interface do usuário.

6.2 Tela de Favoritos e Gerenciamento de Análises

- **Exibição da Biblioteca:** Apresentação dos jogos e análises previamente cadastrados na tabela **`tabela_favoritos`**.
- **Cadastro de Novos Registros:** Abertura e preenchimento do formulário de cadastro, viabilizando o salvamento de atributos como título, status, avaliação numérica, data e resenha textual.
- **Consulta e Edição:** Filtragem dinâmica dos jogos cadastrados e atualização das informações de registros existentes por meio das operações mapeadas na **`FavoritoDao`**.
- **Exclusão de Registros:** Deleção de um item do banco de dados com atualização imediata do estado visual da tela.

6.3 Validação da Persistência de Dados e do Ciclo de Vida

- **Retenção de Estado:** Comprovação da integridade das informações após o encerramento e reinicialização do aplicativo ou diante de alterações de orientação da tela do dispositivo. Esse comportamento valida a eficácia do gerenciamento de estado

via **`AndroidViewModel`** e a persistência assíncrona garantida pela biblioteca Room.

Para ilustrar de forma detalhada o fluxo de navegação, a responsividade da interface e a persistência dos dados descritas nesta seção, foi gravada uma demonstração em vídeo. O registro audiovisual completo do aplicativo PlayDex em funcionamento pode ser executado a seguir:

Vídeo geral do Projeto.mp4

7 DIFICULDADES ENCONTRADAS E RESOLUÇÃO DE PROBLEMAS

Durante as etapas de desenvolvimento do aplicativo PlayDex, a equipe enfrentou desafios técnicos específicos relacionados à reestruturação prévia da arquitetura da aplicação e ao gerenciamento do processamento assíncrono durante a integração do banco de dados local.

7.1 Gerenciamento de Estado e Ciclo de Vida: Transição para *AndroidViewModel*

A principal dificuldade técnica encontrada antecedeu a implementação da biblioteca Room. No estágio do Trabalho 2, tanto a Tela de Pesquisa quanto a Tela de Favoritos não possuíam o componente *ViewModel* integrado, dependendo exclusivamente de estados locais e temporários em memória (**`mutableStateListOf`** e **`mutableStateOf`**).

Em razão dessa abordagem, eventos de mudança de configuração no dispositivo — como a rotação da tela — provocavam a destruição e a recriação da *Activity*. Esse comportamento do ciclo de vida da plataforma Android reinicializava a interface do usuário e causava a perda imediata de todas as informações exibidas.

Diante desse cenário e reconhecendo a *ViewModel* como um pré-requisito arquitetural para a persistência de dados, a equipe reestruturou o código-fonte de ambas as telas. Foram desenvolvidas as classes **`PesquisaViewModel`** e **`FavoritosViewModel`** estendendo **`AndroidViewModel`**. Essa abstração permitiu encapsular o estado dos dados, assegurando a retenção das informações perante as alterações do ciclo de vida da interface. Ademais, a herança de **`AndroidViewModel`** viabilizou o acesso ao contexto da aplicação (**`Application`**), recurso necessário para a posterior instanciação do banco de dados via **`AppDatabase.getDatabase(application)`**.

7.2 Adequação ao Fluxo Assíncrono (*Dispatchers.IO* e *Dispatchers.Main*)

Após o desacoplamento da camada de apresentação, a integração com o Room introduziu a necessidade de gerenciar o processamento assíncrono das operações de leitura e escrita. A biblioteca proíbe a execução de instruções de entrada e saída (*I/O*) na *thread* principal de interface (*UI Thread*); contudo, o Jetpack Compose exige que as modificações no estado reativo da tela ocorram obrigatoriamente na *Main Thread* para permitir a re-renderização visual sem falhas de execução.

A equipe solucionou o gerenciamento de *threads* estruturando corrotinas por meio da instrução **`viewModelScope.launch(Dispatchers.IO)`** no âmbito das *ViewModels*, direcionando as chamadas aos DAOs (**`pesquisaDao.buscarItens()`**, **`inserirFavorito()`**, **`atualizarFavorito()`**) para execução em segundo plano. Sequencialmente, aplicou-se o contexto de execução **`withContext(Dispatchers.Main`**) para retornar os dados processados à *thread* principal com segurança, viabilizando a atualização dos estados reativos da interface.

8 CONSIDERAÇÕES FINAIS

A realização deste trabalho permitiu consolidar a transição do aplicativo PlayDex de um estado de protótipo temporário para uma aplicação móvel com persistência de dados local robusta e arquitetura bem definida. A substituição das listas voláteis em memória pelo banco de dados SQLite via biblioteca Room resolveu de forma definitiva a perda de informações ao encerrar a aplicação ou ao alterar a orientação do dispositivo, atendendo às exigências de usabilidade do sistema.

A reestruturação arquitetural prévia com o uso de **`AndroidViewModel`** demonstrou-se um passo decisivo, pois permitiu o desacoplamento entre a camada de interface do usuário (*UI*) e a lógica de negócios, além de viabilizar a injeção do contexto da aplicação necessário para a instanciação do banco de dados **`AppDatabase`**. Adicionalmente, o controle explícito de assincronismo com *Coroutines* e a alternância de contextos de execução (**`Dispatchers.IO`** para operações de entrada/saída e **`Dispatchers.Main`** para atualização da interface) asseguraram a fluidez da aplicação sem causar travamentos na *thread* principal.

Como sugestões para trabalhos e pesquisas futuras, recomenda-se a expansão do esquema do banco de dados para suportar relacionamentos complexos (como tabelas de categorias de jogos ou usuários), a implementação de um sistema de migração de banco de dados (*Room Migrations*) automatizado para novas versões, e a integração com APIs externas de dados de jogos para enriquecimento automático dos registros salvos localmente.